Singly Linked List – Traverse List ReverseOrderIterative and Time Complexity.

Traversing List ReverseOrder means traversing list starting from last index to first index.

Program:

```
void TraverseListReverseOrderIterative(Node **head)

{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    reverseList(head);
    Node **current = head;
    while (*current != nullptr)
    {
        cout << (*current)->data << " ";
        current = &((*current)->next);
    }
    cout << endl;
    reverseList(head);
}
....

        case 24: {

            cout << endl;
            cout << endl;
            cout << "Traverse List Reverse Order Iterative" << endl;
            TraverseListReverseOrderIterative(&head);
            cout << endl;
            cout << endl;
```

```
            break;
        }
```

Function Call:

$Function\ call\ occurs\ when\ Case\ 24\ is\ initiated\ from\ Switch - Case$:

$TraversingListReverseOrderIterative(\&head);$

$There\ are\ two\ criteria\ of\ input\ Linked\ List$:

1)*Linked List is Empty*:

$Pictorially\ depicted\ as$:

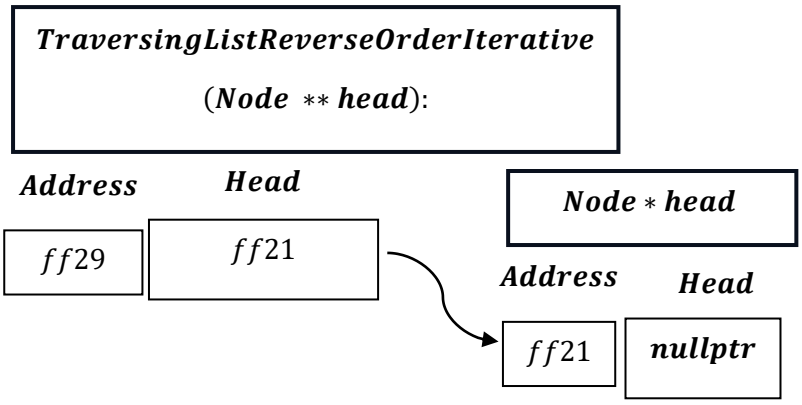
$Node * head$

$Address$	$Head$
$ff21$	$nullptr$

```
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
```

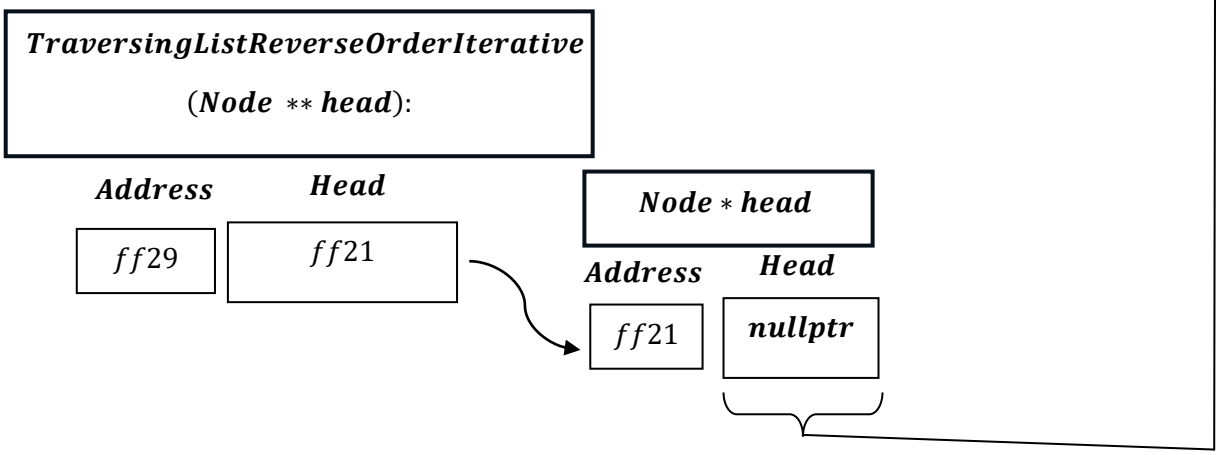
$TraversingListReverseOrderIterative(\&head: ff21);$

```
void TraverseListInorder2(Node **head) {….}
```



$if(*head == nullptr)$ *if it is true*

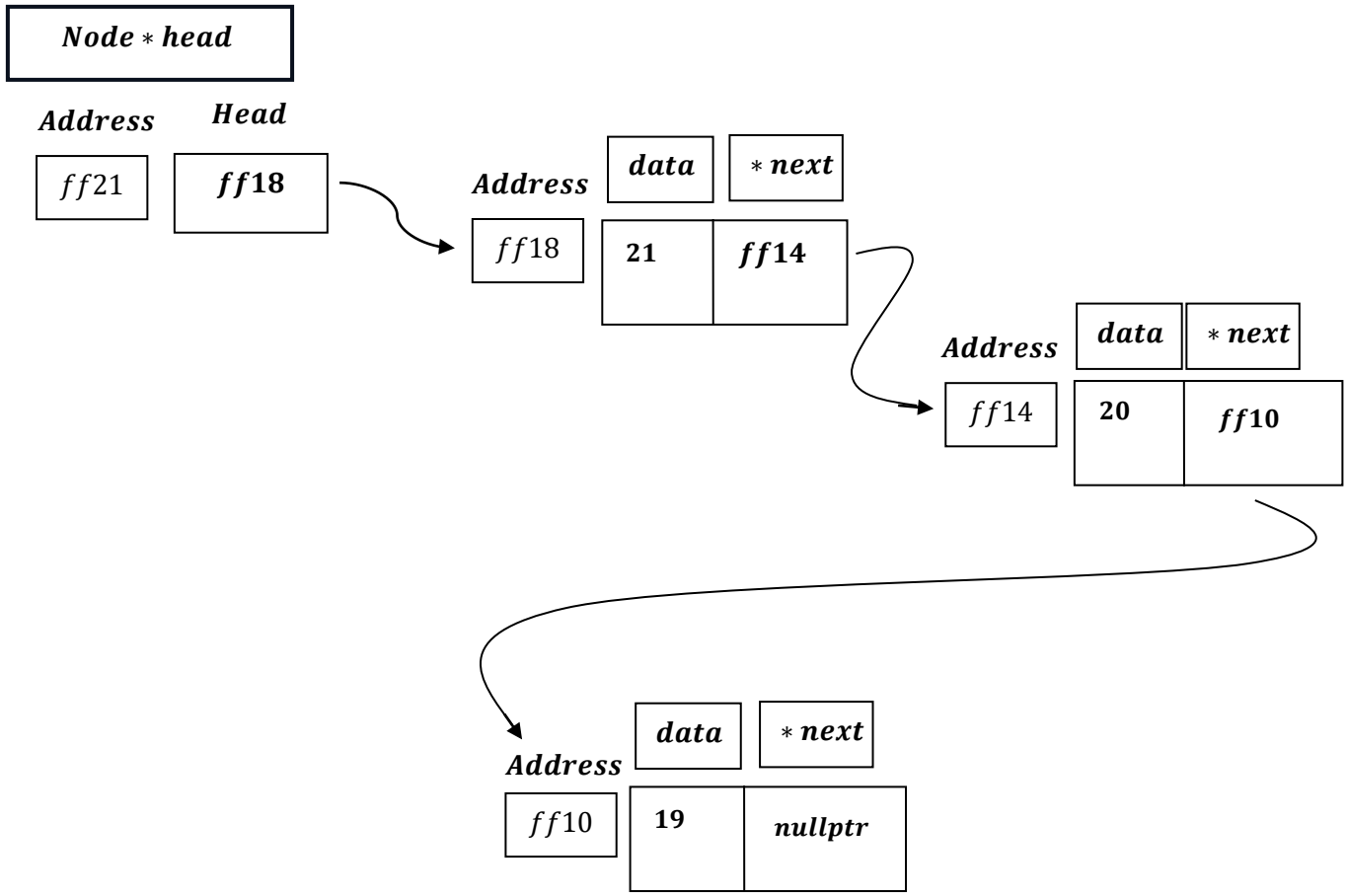
If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the `TraversingListReverseOrderIterative` function.



If List is Empty , then , it prints a message : `Error: List is empty.` and returns nothing as function has void data type . And function ends.

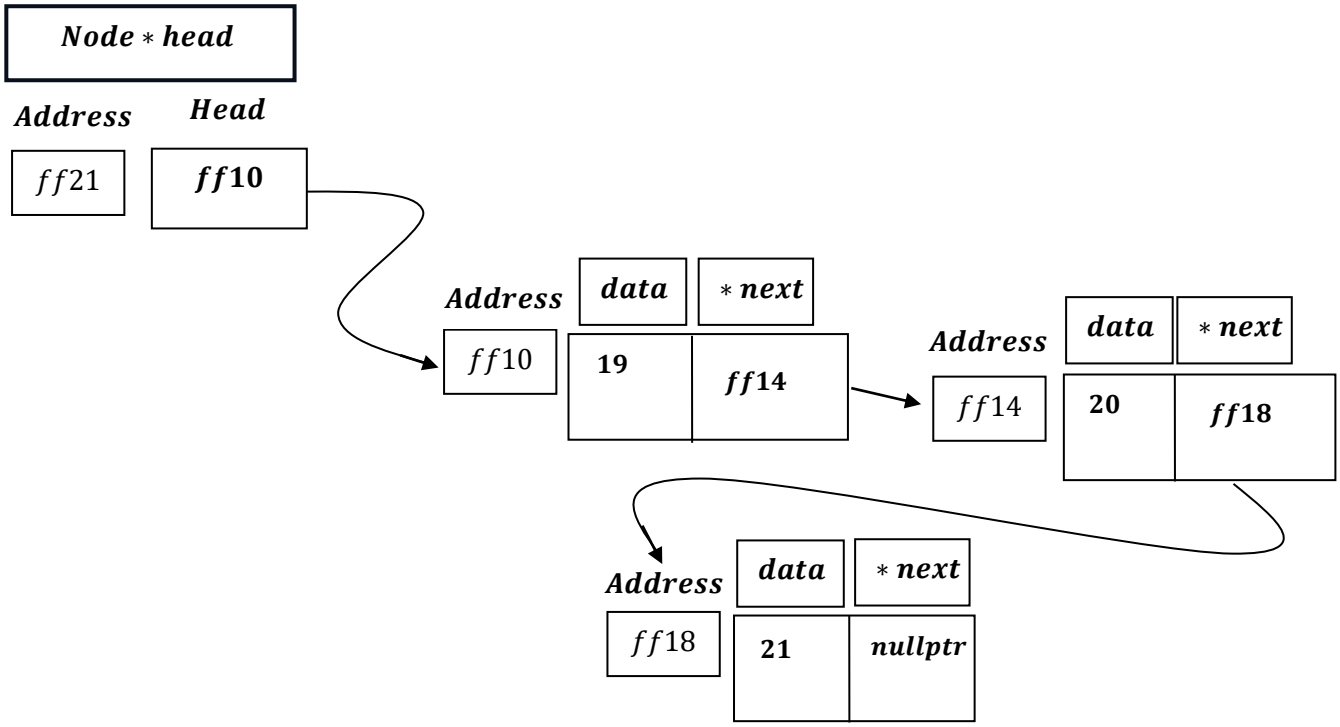
---

2)*Linked List is not Empty*:

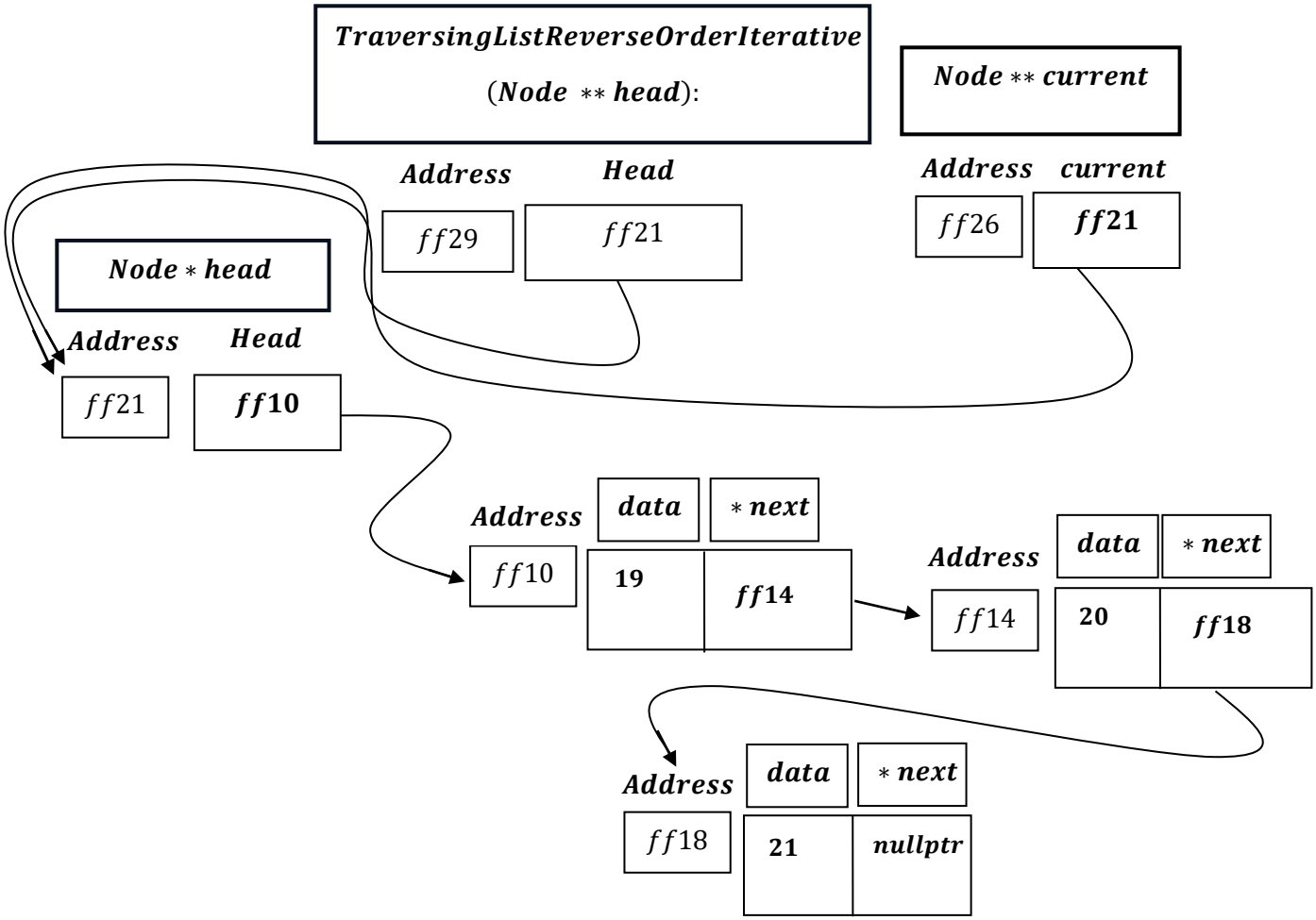



```
reverseList(head);
```

After reversing list we get:



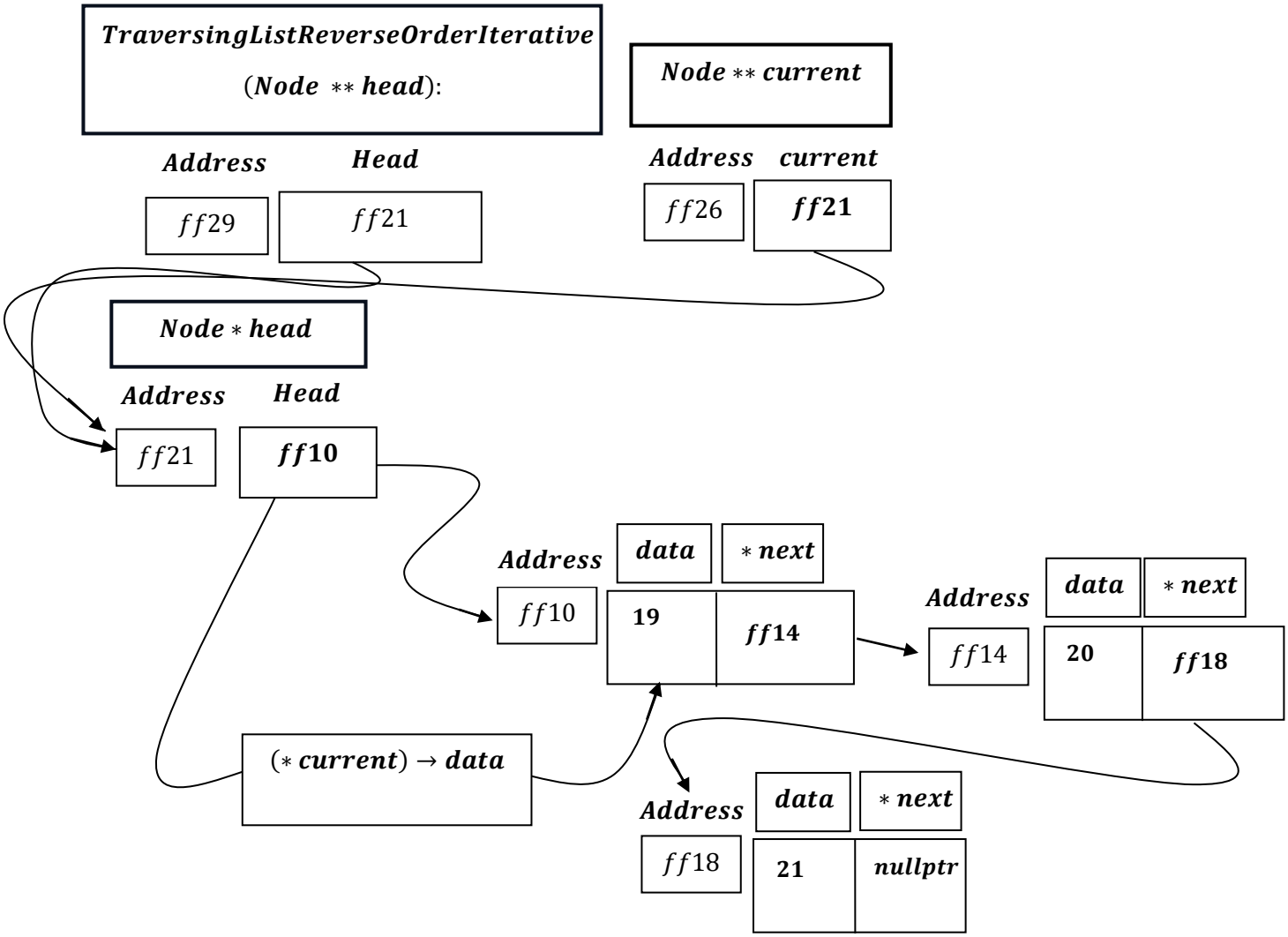

```
Node **current = head;
```



```
while (*current != nullptr)
    {
        cout << (*current)->data << " ";
        current = &((*current)->next);
    }
```

$*$ ***current is not equal to*** $(!=)$ ***nullptr which is true then***:

print $\rightarrow$ $(*$ ***current***$)$ $\rightarrow$ ***data*** , ***which is*** : **19**.



$(*$ ***current***$)$, ***here braces prioritize the pointer current variable, i.e.***

Node $*$ $\underline{*\ current}$ ***pointer, and dereference it , to get the***

Node $*$ ***it points to. Here*** $\underline{Node\ *}$ ***points to address***: ff**10**, ***where***

$\underline{current}$ points to address : $ff21$. Hence as $\underline{*current}$ pointergets deferenced by $*$ (dereferencing operator), i.e. now

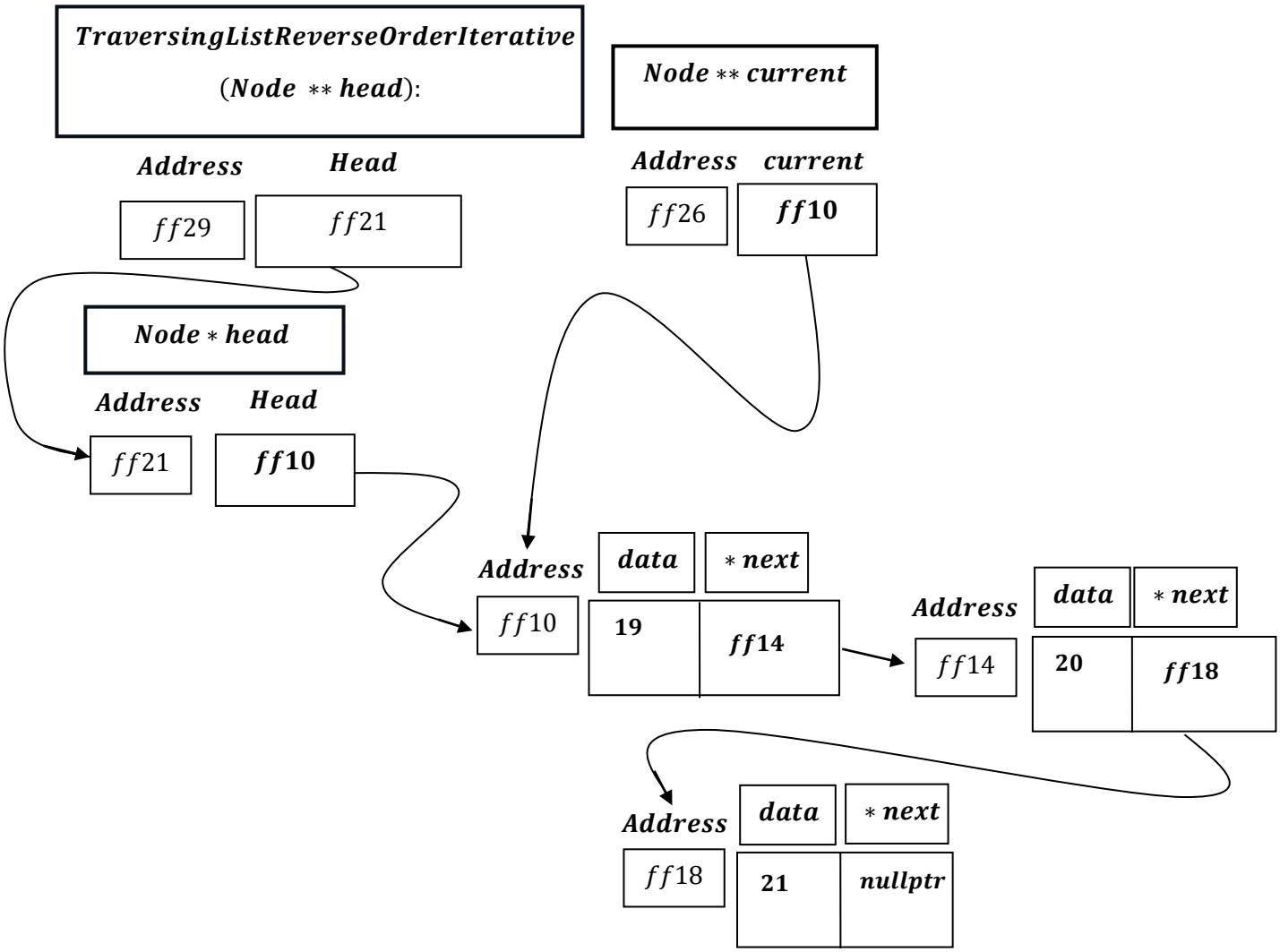
no longer pointing to address : $ff21$, hence , now

Node $*$ pointed address is referenced i.e. $ff10$ and it gives us `data`

stored in address $ff10$ i.e., 19.

Now updation,

```
        current = &((*current)->next);
```



During the first iteration, ($*$ current)dereferences current to get the Node $*$ it points to, which is the first node in the list. The expression ($*$ current) $\rightarrow$ next , where current points to Node $*$ Head : $ff21$, but ($*$ current) dereferences first and now Node $*$ points to first node, hence ($*$ current) $\rightarrow$ next, will point to next pointer

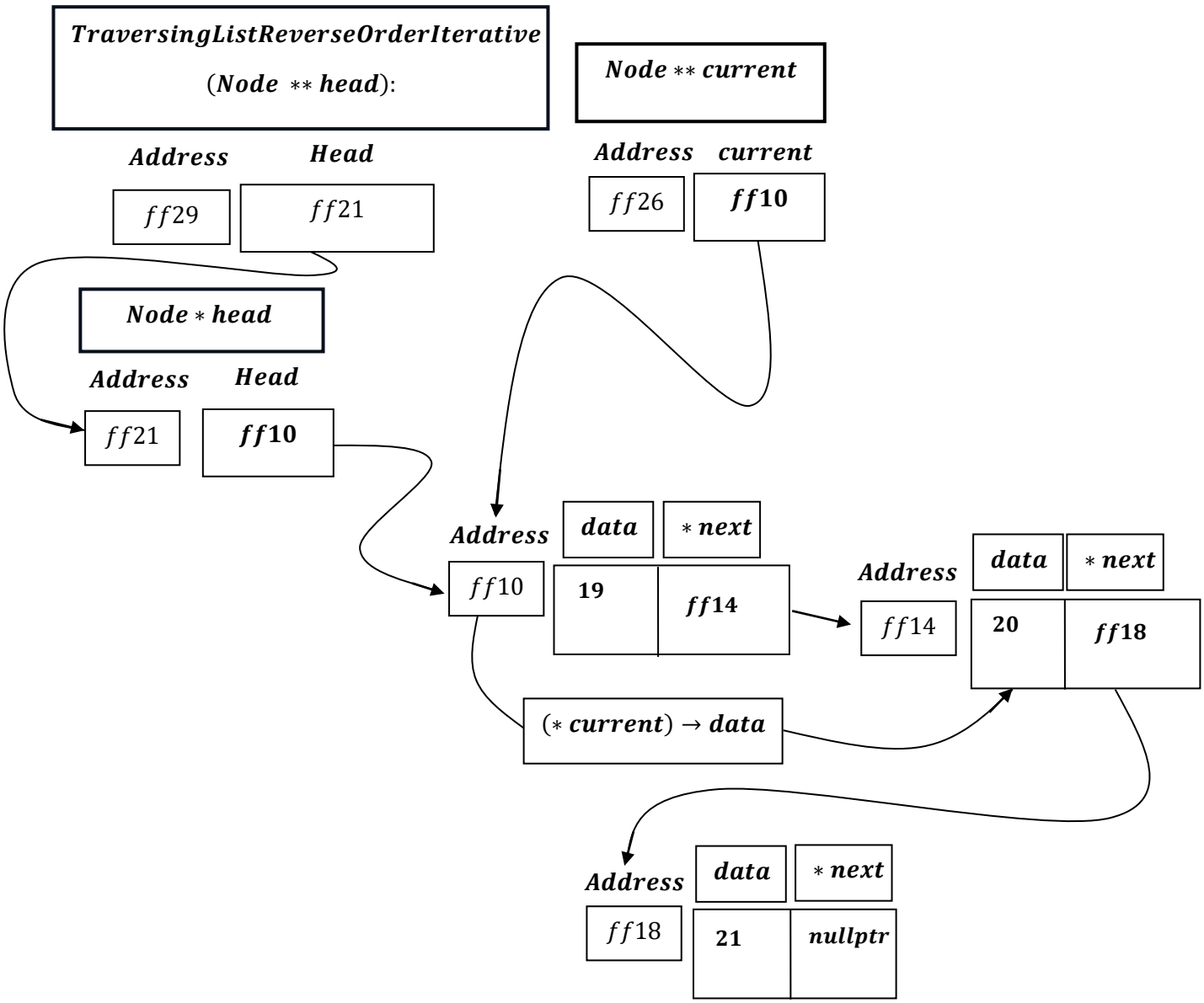
$of\ first\ node\ i.e.\ ff14$ $.The\ \&\ operator\ then\ takes\ the\ address\ where\ this$

$next\ pointer\ is\ stored\ i.e.\ the\ address: ff10, where\ ff14\ is\ stored\ .Hence,$

$current = \&((*current) \rightarrow next) = ff10\ under\ which\ ff14\ is\ stored.$

$*current\ is\ not\ equal\ to\ (!=)\ nullptr\ which\ is\ true\ then:$

$print \rightarrow (*current) \rightarrow data\ , which\ is : 20.$



$(*\ current)$, here braces prioritize the pointer current variable, i.e.

Node $*$ $\underline{*\ current}$ pointer, and dereference it, to get the

Node $*$ it points to. Here $\underline{Node *}$ points to address: $ff14$, where

$\underline{current}$ points to address : $ff10$. Hence as $\underline{*\ current}$ pointergets deferenced by $*$ (dereferencing operator), i.e. now

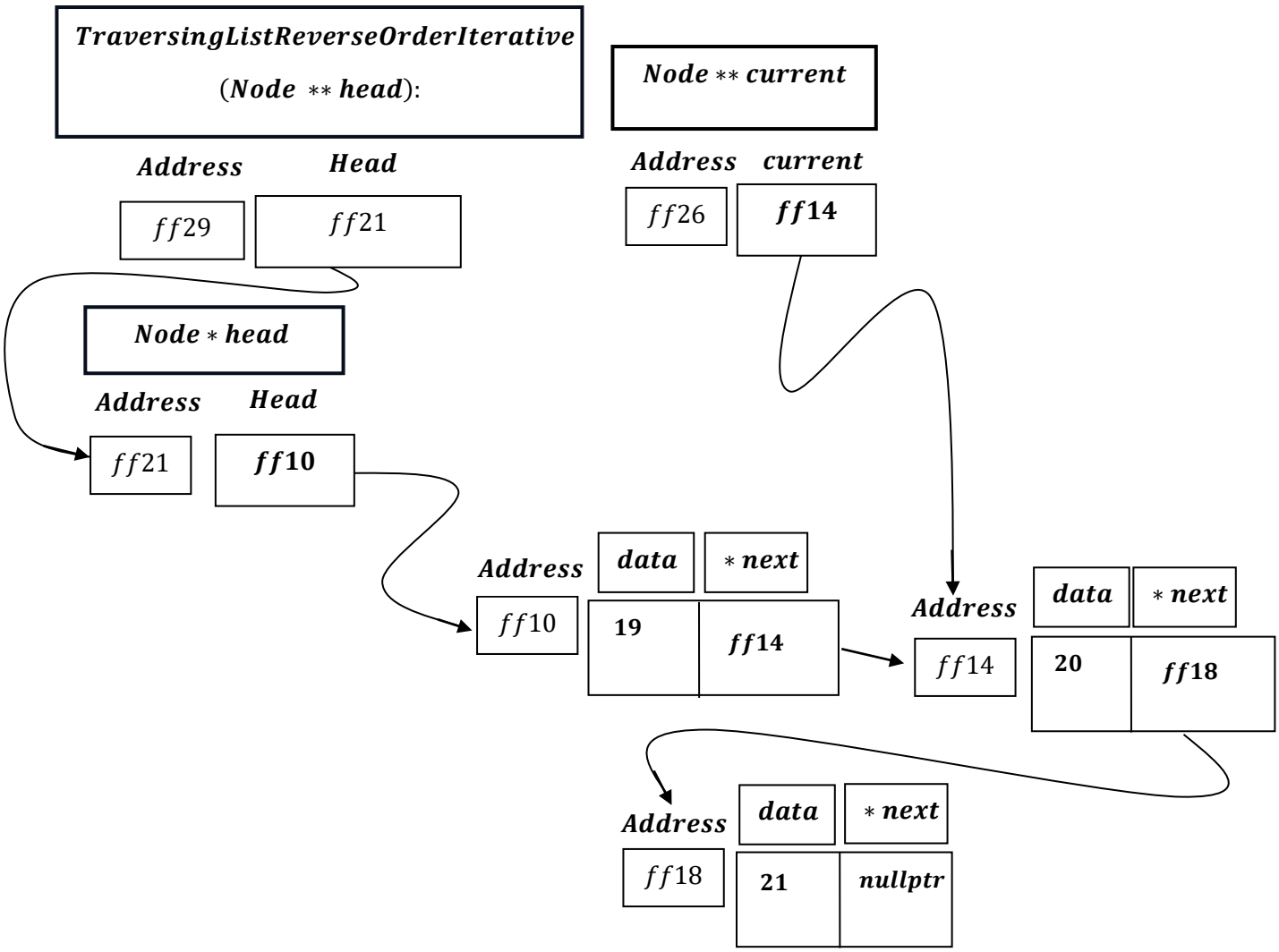
no longer pointing to address : $ff10$, hence, now

Node $*$ pointed address is referenced i.e. $ff14$ and it gives us `data`

stored in address $ff14$ i.e., 20.

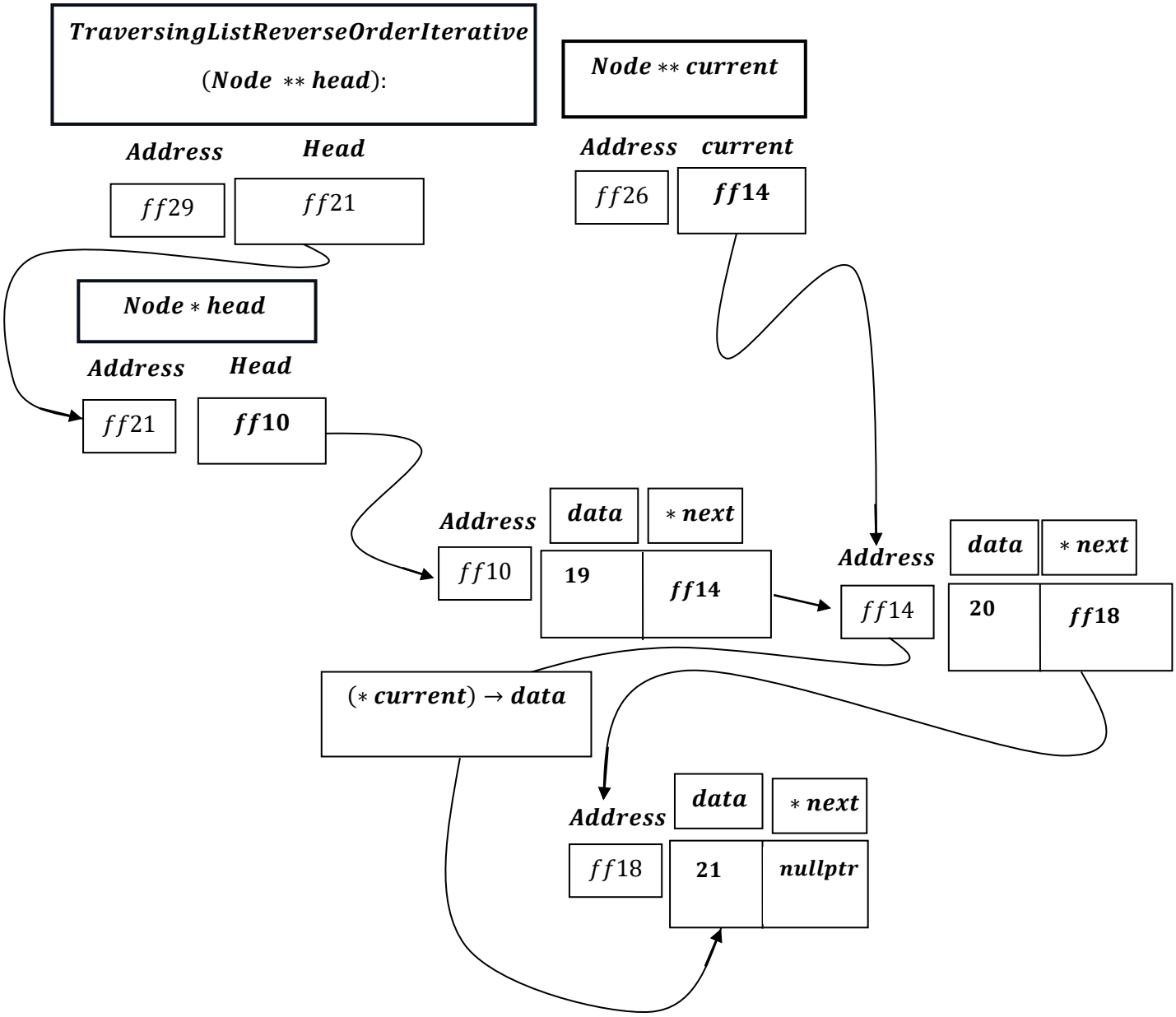
Now updation,

```
        current = &((*current)->next);
```



$During\ the\ second\ iteration, (*current) dereferences\ current\ to\ get\ the\ Node\ *$
$it\ points\ to, which\ is\ the\ second\ node\ in\ the\ list. The\ expression\ (*current) \rightarrow next$
$, where\ current\ points\ to\ address: ff10, but\ (*current)\ dereferences\ first\ and$
$now\ Node * points\ to\ second\ node, hence\ (*current) \rightarrow next, will\ point\ to$
$next\ pointer\ of\ second\ node\ i.e.\ ff18\ .The\ \&\ operator\ then\ takes\ the\ address\ where$
$this\ next\ pointer\ is\ stored\ i.e.\ the\ address: ff14, where\ ff18\ is\ stored\ .Hence,$
$current = \&\big((*current) \rightarrow next\big) = ff14\ under\ which\ ff18\ is\ stored.$

$*current\ is\ not\ equal\ to\ (!=)\ nullptr\ which\ is\ true\ then:$
$print \rightarrow (*current) \rightarrow data\ , which\ is: 21.$



$(*\ current)$, here braces prioritize the pointer current variable, i.e.

Node $*$ $\underline{*\ current}$ pointer, and dereference it, to get the

Node $*$ it points to. Here $\underline{Node *}$ points to address: $ff18$, where

$\underline{current}$ points to address : $ff14$. Hence as $\underline{*\ current}$
pointer gets deferenced by $*$ (dereferencing operator), i.e. now

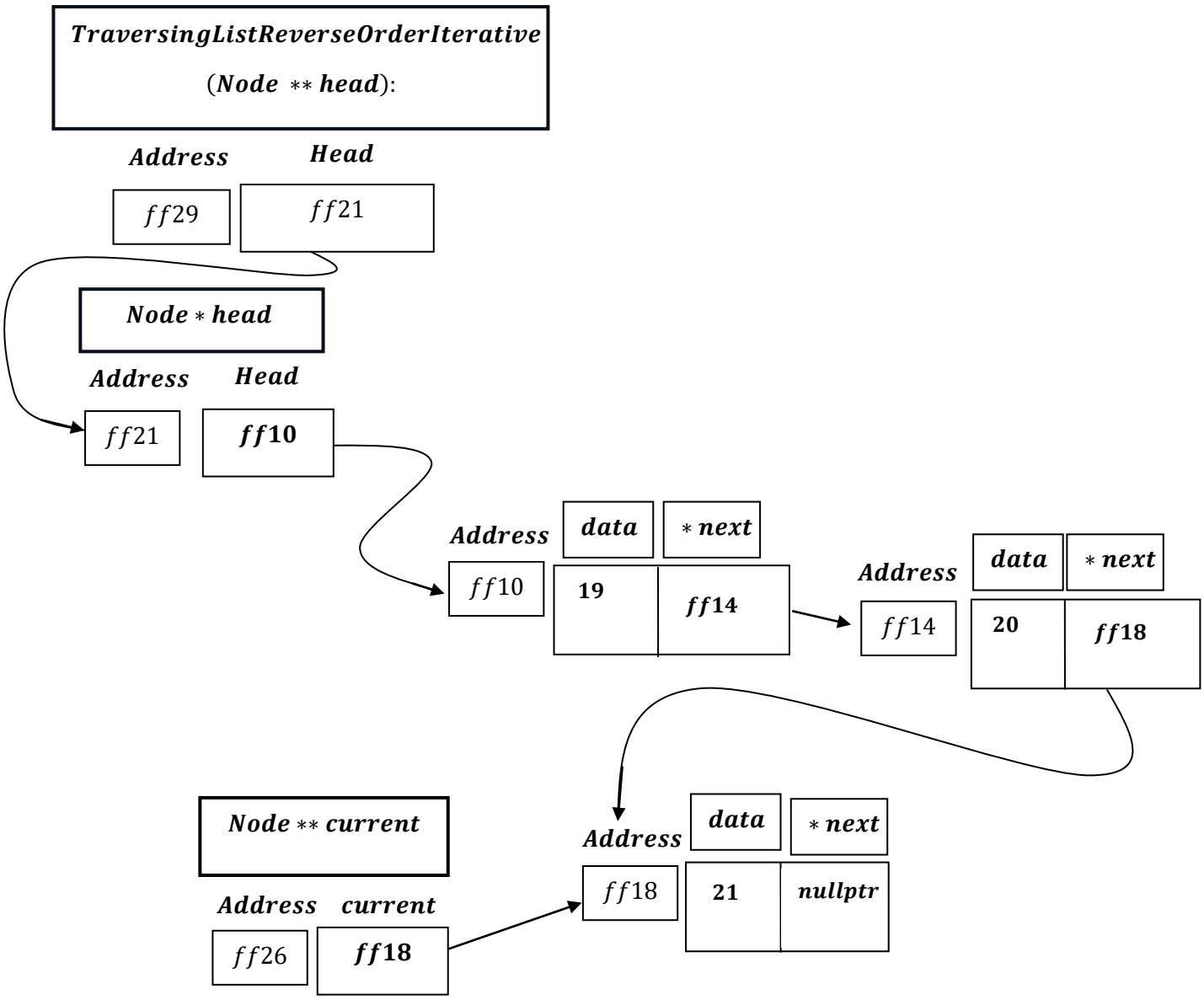
no longer pointing to address : $ff14$, hence, now

Node $*$ pointed address is referenced i.e. $ff18$ and it gives us `data`

stored in address $ff18$ i.e., 21.

Now updation,

```
        current = &((*current)->next);
```



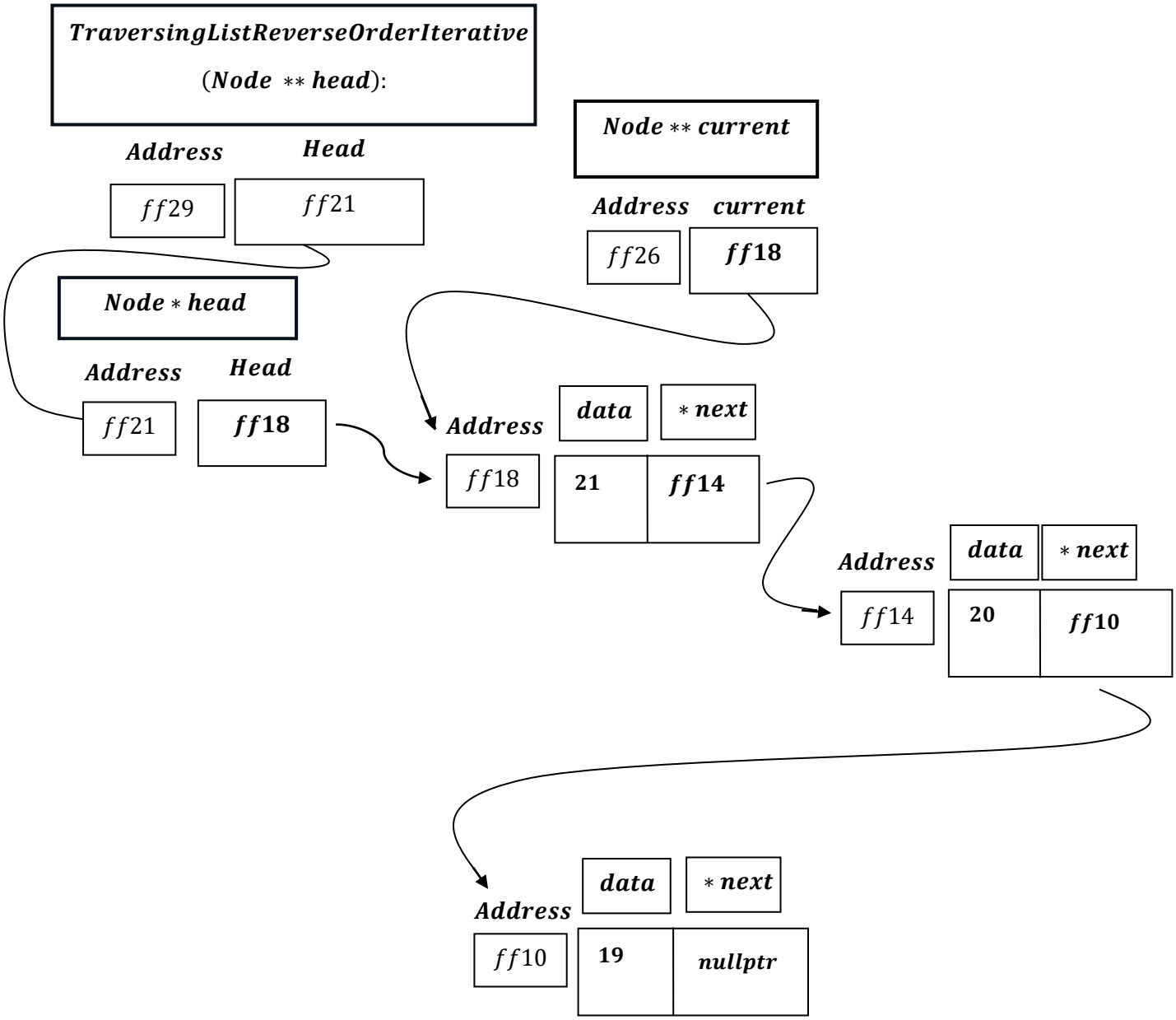
During the third iteration, (∗ current) dereferences current to get the Node ∗ it points to, which is the third node in the list. The expression (∗ current) → next, where current points to address : ff18, but (∗ current) dereferences first and now Node ∗ points to third node, hence (∗ current) → next, will point to next pointer of third node i.e. ``nullptr`` . The & operator then takes the address where this nullptr is stored i.e. the address: ff18, where ``nullptr`` is stored . Hence, current = &((∗ current) → next) = ff18 under which ``nullptr`` is stored.

```
while (*current != nullptr) {…} //condition checking
```

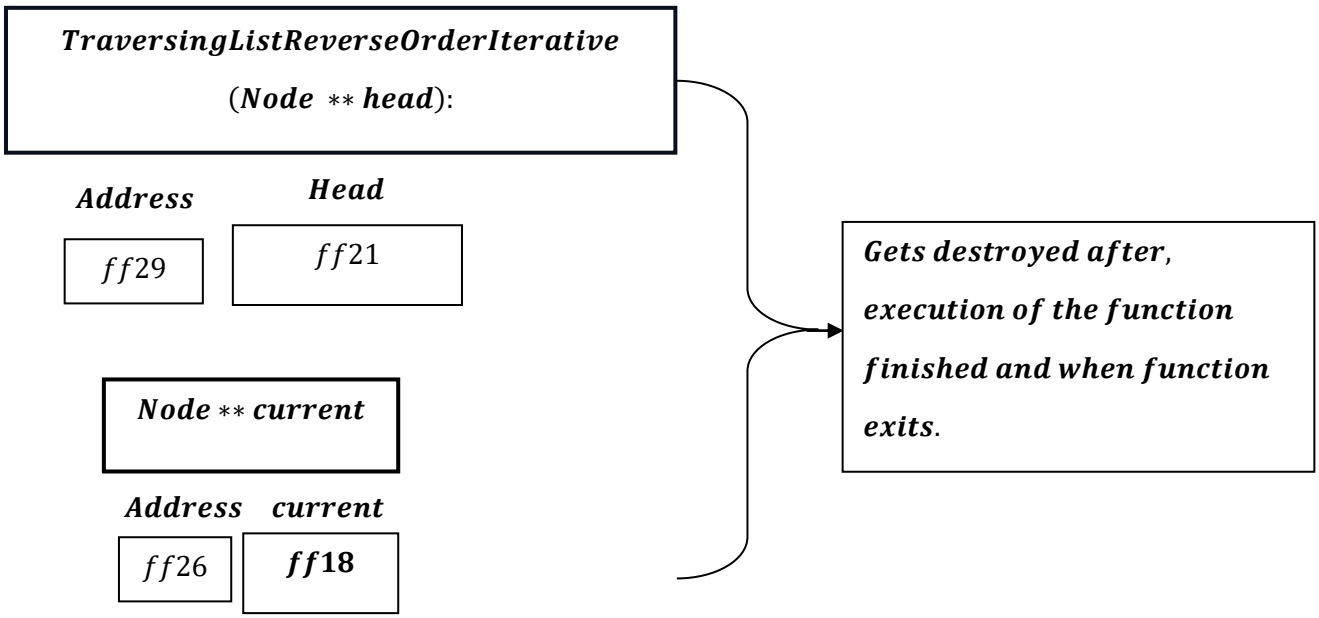
∗ current, due to conditional checking, hence no braces are needed to prioritize the pointer current variable, i.e. Node ∗ ∗ current pointer, and dereference it , to get the Node ∗ it points to. Here Node ∗ points to ``nullptr``, where current points to address : ff18. Hence as ∗ current pointergets deferenced by ∗ (dereferencing operator), i.e. now no longer pointing to address : ff18, hence , now Node ∗ pointed address is referenced i.e. ``nullptr``.

Now, as soon as we get ``nullptr``, the condition of while loop fails and exits from the loop.

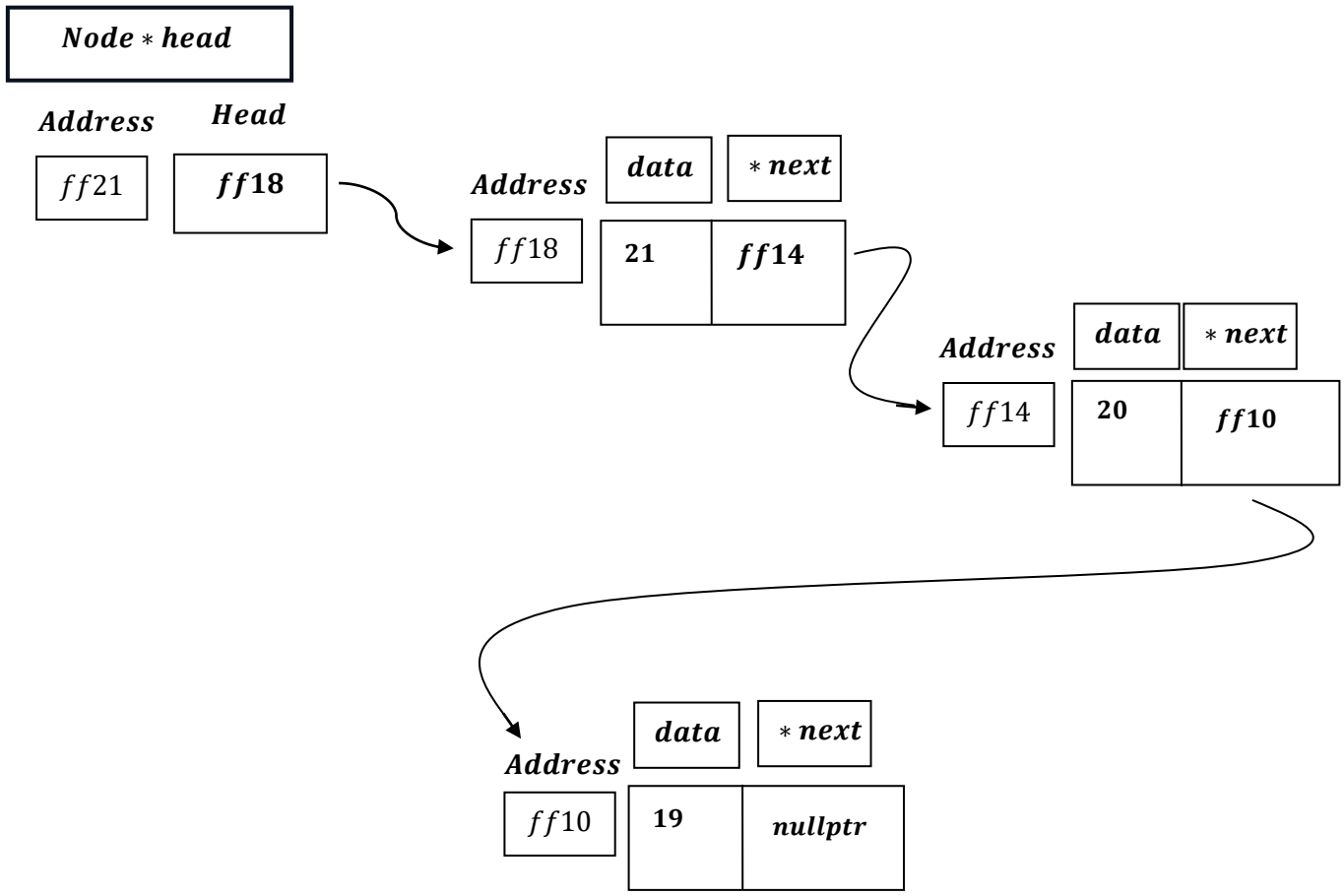
```
    reverseList(head);
```



Now, as the function get finished, all temporary variable created earlier in process for the program will get destroyed.



And we are left with:



Now, `TraversingListReverseOrderIterative` step by step explanation :

1. ***Check if the List is Empty***:

    - *The function first checks if the list is empty by verifying if the head pointer is nullptr. If it is, it prints* List is empty. *and exits the function.*

2. ***Reverse the List***:

    - *The function calls reverseList(head), which is assumed to reverse the linked list. This step is crucial because the function aims to traverse the list in reverse order.*

3. ***Initialize Current Pointer***:

    - *A pointer current is initialized to point to the head of the list. This pointer will be used to iterate through the list.*

4. ***Traverse and Print the List***:

    - *The function enters a loop that continues until current points to nullptr.*
    - *Inside the loop, it prints the data of the current node.*
    - *It then updates current to point to the next node in the list.*

5. ***Reverse the List Back to Original Order***:

- *After printing the list in reverse order, the function calls reverseList(head) again to restore the list to its original order.*

Time Complexity of

TraversingListReverseOrderIterative

```
void TraverseListReverseOrderIterative(Node **head)

{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    reverseList(head);
    Node **current = head;
    while (*current != nullptr)
    {
        cout << (*current)->data << " ";
        current = &((*current)->next);
    }
    cout << endl;
    reverseList(head);
}
```

$O(1)$

$O(n)$

$O(1)$

runs `n` times.

runs `n + n` = 2n times.
i.e. $O(n)$.

$O(1)$

$O(n)$

<u>*Best Case (When List is Empty)*</u>

When List is Empty, then, as soon as it enters the function: TraversingListReverseOrderIterative, if checks whether head pointer pointed to ``nullptr`` or not , if so then outputs: ``List Is Empty`` and returns nothing , and then exits from function. This takes, constant `c` amount of time i.e. $\Omega(1)$.

Hence when List is Empty, we get our best case : $\Omega(1)$.

$\underline{Worst\ Case\ (When\ List\ Is\ Not\ Empty)}$

$When\ list\ is\ not\ empty\ , it\ traverse\ whole\ list\ , assuming\ `n`\ is\ the\ length\ of\ the\ List.$

$\rightarrow Head\ doesnot\ point\ to\ nullptr\ , when\ List\ is\ not\ empty, hence\ checking\ takes\ O(1)\ time\ complexity.$

$\rightarrow Reverse\ List\ function\ takes\ O(n)\ time\ complexity.$

$\rightarrow Then\ the\ loop\ traverse\ `n`\ no.of\ nodes\ present\ in\ the\ list.$

- $Where\ statement : cout \ll (*current) \rightarrow data \ll ``\ ``; outputs\ , `n`\ times.$
- $Assignment: current = \&((*current) \rightarrow next); this\ assignment\ takes\ `n`\ times.$

$Hence\ total\ number\ of\ statements\ in\ the\ loop\ runs\ `n`\ times = O(n+n) = O(2n)$
$= 2 \times O(n) = O(n)$.

$\rightarrow Then\ again, Reverse\ List\ function\ takes\ O(n)\ time\ complexity.$

$$Hence\ , total\ time\ complexity: -O(1) + O(n) + O(n) + O(n) = O(n).$$

$\underline{Average\ Case(Average\ Case = Worst\ Case)}$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, including\ running\ function\ reverseList(head)\ function\ twice, the\ time\ complexity\ is\ (O(n))$
$, where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$The\ average\ case\ for\ traversing\ a\ linked\ list\ typically\ involves\ traversing\ a\ list\ of\ average\ length. Since\ the\ function\ must\ visit\ each\ node\ to\ print\ its\ data, the$

$time\ complexity\ is\ (O(n)), where\ (n)\ is\ the\ number\ of\ nodes\ in\ the\ list.$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq n \leq c_2 \times n$

$Singly\ Linked\ List$	$Cases$	$Time\ Complexity$
$TraversingListReverseOrderIterative$	$1. Best\ Case$	$\Omega(1)$
	$2. Worst\ Case$	$O(n)$
	$3. Average\ Case = Worst\ Case$	$\Theta(n)$

****